arXiv:2611.10469v1 [cs.CL] 7 Nov 2026

HIBANA: FAST AGENTIC INFERENCE ON UNIFIED-MEMORY DEVICES

Yahya Emara [*1] **Chi-Chih Chang** [*1] **Anthony Fei** [1] **Ahmed F AbouElhamayed** [1] **Chong-Yan Chen** [2]
Hung-Yueh Chiang [3] **Kai-Chiang Wu** [4] **Mohamed S. Abdelfattah** [1]

https://sharar-project.github.io

ABSTRACT

Local AI promises capable personal assistants that run on users' own devices, keeping data and computation under their control. Open-weight models that fit on local devices can now handle practical tasks such as coding. Yet current inference engines fall short. Popular engines such as vLLM and SGLang target datacenter GPUs with dedicated host memory, while local engines underuse the memory hierarchy of unified-memory devices. We present Hibana, an inference engine for unified-memory devices that co-optimizes model representation, memory management, and decoding strategy. **Model surgery**, the process of editing a trained model's architecture to suit its hardware, changes how the model computes, shrinking its memory footprint and data movement. **Unified-memory management** preserves each agent's state across turns, so long-running agents can resume instantly instead of recomputing their state. **Adaptive speculation** generates several tokens per step. Hibana codesigns these features around the shared memory capacity and limited bandwidth of unified-memory systems-on-chip to support local multi-agent workloads. On a single DGX Spark serving Qwen and Nemotron models of up to 125B parameters, Hibana completes up to 3.4× more agentic tasks per hour and delivers up to 3.9× higher decoding throughput than other established frameworks such as vLLM, NVIDIA Flare, llama.cpp, and TensorFold. Code is available at https://github.com/sharar-project/hibana.

1 INTRODUCTION

Large language models have evolved from conversational assistants into systems capable of writing code and executing multi-step agentic tasks. Driven by scaling laws (Kaplan et al., 2020), frontier models have grown to trillions of parameters, making datacenter-scale infrastructure the dominant way to serve them. Running capable models locally would keep user data under their control and eliminate metered API inference charges. Recent progress in both model and hardware suggests that this is becoming practical. Figure 1 shows that Qwen3.8-Flash-Next surpasses GPT-5.5 on the Artificial Analysis Intelligence Index (Artificial Analysis, 2026b) while fitting within the 128 GB memory of a single NVIDIA DGX Spark. Moreover, an open-weight model matching GPT-5.5's capability reached this memory budget approximately four months after GPT-5.5's release.

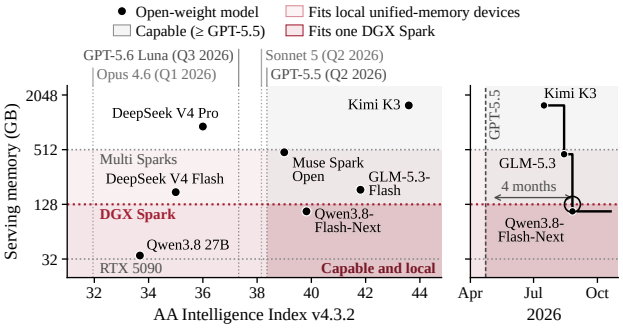

Figure 1. Open-weight models' capability and serving memory. **Left** Qwen3.8-Flash-Next scores above GPT-5.5 and fits on one DGX Spark. (**Right.**) The smallest model at GPT-5.5's level *fits* on one DGX Spark ∼4 months after GPT-5.5 was released.

This progress is driven by improvements in model architecture and training efficiency. Efficient architectures, including mixture-of-experts and sparse or linear attention, reduce inference costs (Fedus et al., 2022; Child et al., 2019; Yang et al., 2025; Dao & Gu, 2024), while knowledge distillation (Hinton et al., 2015; Gu et al., 2024) and low-precision formats such as `NVFP4` and `MXFP4` further reduce serving requirements. Combined with increasingly capable local hardware, these advances bring LLM-powered agents within reach of local devices. However, fitting a capable model into memory does not guarantee efficient execution of agentic workloads, because decoding is limited by memory band-

[*]Equal contribution [1]Cornell University [2]Department of Computer Science, University of Illinois Urbana-Champaign [3]Independent Researcher [4]National Yang Ming Chiao Tung University. Correspondence to: Yahya Emara <ye52@cornell.edu>, Chi-Chih Chang <cc2869@cornell.edu>.

Preprint.

width, which unified-memory devices trade for capacity.

Traditional local inference has largely focused on single-user chat, often with one active decoding stream. Modern agents, however, invoke tools and spawn subagents, generating concurrent requests while maintaining persistent sessions across turns. This concurrency creates opportunities for batching and amortizing model-weight reads, which is particularly important on unified-memory devices with limited memory bandwidth. Meanwhile, persistent sessions compete with active requests for the same memory, and unified-memory devices have no separate host-DRAM tier to move them to.

Existing inference systems address these requirements only partially. llama.cpp (llama.cpp Contributors, 2026a) supports extensive sub-4-bit quantization but exhibits limited throughput scaling with increasing batch size in our measurements. vLLM and SGLang (Kwon et al., 2023; Zheng et al., 2024) primarily target datacenter GPUs, while recent local systems such as NVIDIA Flare (NVIDIA, 2026a) and TensorFold (TensorFold Contributors, 2026) support agentic workloads but leave opportunities to improve expert-weight reuse and session management.

We present **Hibana**, an inference engine designed for concurrent agentic workloads on unified-memory devices. Hibana jointly optimizes model representation, memory management, and decoding to reduce memory traffic per accepted token. The contributions of Hibana are threefold:

- **Model surgery.** We define *model surgery* as editing a trained model's architecture to suit its hardware. Specifically, we introduce expert sharing, which selects a shared set of experts per layer for batched decoding and verification, and a two-bit draft head with FP8 rescoring, which cuts each draft step's head read from about 640 MB to about 180 MB. We also reduce model-weight traffic by encoding routed experts and dense-layer decode copies as three-bit FP8-codebook indices for FP8 tensor-core execution.
- **Unified memory management.** We design an *SSD session cache* that preserves agents' KV, recurrent, and draft states across turns using a small, fixed staging buffer. INT8 recurrent states shrink every session Hibana keeps, and *Engram offloading* serves large embedding tables from the SSD.
- **Adaptive speculation.** We adapt MTP draft lengths using recent acceptance rates, subject to batch-level execution constraints (adaptive drafting), and adapt the number of diffusion-drafted tokens verified on dense models (adaptive verification). We also optionally accept draft tokens that the target ranks near its top prediction (relaxed verification), bounding the amplification of next-token probabilities.

We evaluate Hibana on a single NVIDIA DGX Spark using Qwen3.8-27B (Qwen Team, 2026a), Qwen3.8-Flash-Next (Qwen Team, 2026b), Nemotron-3.5-Bolt (NVIDIA, 2026b), and Nemotron-3.5-Super (NVIDIA, 2026c). We measure agentic tasks completed per hour alongside decoding throughput and time to first token. Compared with vLLM, Flare, llama.cpp, and TensorFold, Hibana completes up to $3.4\times$ more agentic tasks per hour and achieves up to $3.9\times$ higher decoding throughput (Figure 2).

2 Background and Challenges

2.1 High Capacity, Limited Bandwidth on DGX Spark

NVIDIA DGX Spark has 128 GB of unified LPDDR5X memory shared by the CPU and GPU. This allows large models to fit in a compact system, but its 256-bit memory interface provides only 273 GB/s of peak bandwidth, far less than the HBM used in datacenter GPUs (NVIDIA, 2026e). This trade-off matters during autoregressive decoding, which generates one token at a time. At small batch sizes, each step must read the weights needed for the next token, so decoding is often limited by memory bandwidth rather than compute.

Two common approaches to this problem are weight quantization and speculative decoding. Both help, but each faces challenges on this hardware.

Challenge 1. Sub-4-bit weights. Quantization reduces memory traffic by storing weights with fewer bits. Going below 4 bits cuts traffic further but makes model quality harder to preserve, and GPUs lack native 2- or 3-bit matrix operations, so weights must be converted to a supported format before each multiply. Without efficient kernels, this conversion can offset the bandwidth savings. Hibana addresses this with calibrated three-bit codebook weights that tensor cores consume as FP8 (§3.1.3).

Speculative decoding takes a different approach: it tries to produce more tokens from each load of the target model's weights. A cheaper draft model proposes several tokens, which the target model verifies together (Leviathan et al., 2023). When multiple tokens are accepted, the cost of reading the target weights is spread across those tokens. Its speedup depends on how many draft tokens are accepted and what drafting and verification cost, and on this hardware both work against it.

Challenge 2. Expert reads during verification. On mixture-of-experts (MoE) models each verified token routes to its own experts (Jiang et al., 2024), so verifying more tokens reads more expert weights. On Qwen3.8-Flash-Next, each token's top five experts alone cover 250 to 460 of a layer's 512 at 16 to 64 requests, so little of the weight reuse that speculation relies on remains. Hibana addresses this

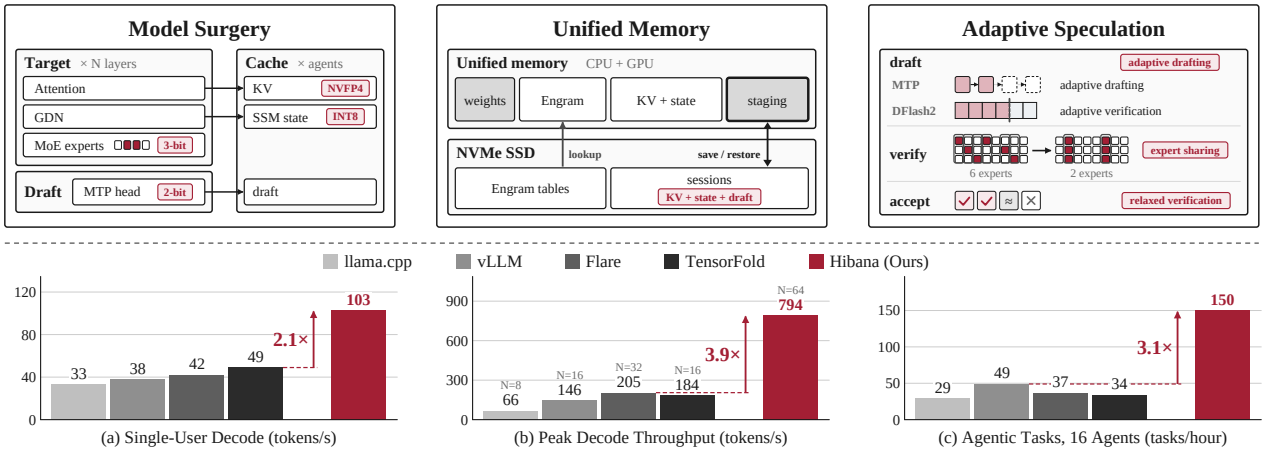

Figure 2. Hibana serving Qwen3.8-Flash-Next on one DGX Spark. (Top.) Hibana's model surgery, unified-memory management, and adaptive speculation (Section 3). (Bottom.) (a) decode speed for a single user, (b) peak decode throughput at each engine's best batch size, and (c) agentic tasks completed per hour with 16 agents. Speedups are over the strongest baseline.

with expert sharing (§3.1.1).

Challenge 3. Drafting cost and varying acceptance. Acceptance varies across prompts and requests, so a fixed draft length either wastes drafting and verification or leaves acceptable tokens undrafted. Drafting is also not free on a bandwidth-bound device, because even a small draft module reads a full-vocabulary output head at every step. Hibana addresses this with adaptive speculation (§3.3) and a two-bit draft head (§3.1.2).

2.2 State Reuse in Agentic Workloads

Agentic workloads create an additional memory challenge. A coding agent may pause while waiting for tool calls, while subagents explore different branches of a shared conversation. These sessions often share an initial prefix but need their own state to resume. Keeping every session in memory limits concurrency, while discarding cached state forces the engine to recompute context when a session returns.

In hybrid attention/recurrent models, resuming a session requires both its attention key-value (KV) cache and a snapshot of each recurrent layer's state at the same token position (Yang et al., 2025). Preserving the draft model's state can also avoid rebuilding its context. Together, these form the cached state needed to resume an agent's *session*.

Challenge 4. Session state in shared memory. On a unified-memory device, moving this state into a CPU buffer does not free capacity, since the CPU and GPU draw from the same memory pool. An SSD offers a separate storage tier, but saving and restoring state introduces I/O latency and requires staging buffers in unified memory. The engine must therefore balance memory used by model weights and active requests against the state saved for future turns. Hibana addresses this with SSD-backed session caching and recurrent-state quantization (§3.2).

3 Methodology

Hibana's optimizations fall into three parts, model surgery (§3.1), unified-memory management (§3.2), and adaptive speculation (§3.3). We design them together rather than in isolation (§3.3 describes where they interact), and Hibana's performance depends on the full set rather than any single component (Figure 8).

3.1 Model Surgery

3.1.1 Expert Sharing

A standard MoE router selects the top-k experts for each token independently, so a batch of R tokens can read up to $\min(E, kR)$ distinct experts from a layer with E experts, and large batches and speculative verification read many expert matrices with little reuse. Hibana instead selects one shared expert set during decode and verification. Each token is guaranteed its top g experts, and the rest of the budget goes to the experts with the most routing mass, the router weight summed over the tokens whose original top-k includes them. Each token then uses up to k of its own candidates that are in the shared set and renormalizes their weights (Figure 3), so it can use an expert selected for another token without adding another expert read. Expert sharing is separate from the model's always-active shared expert.

For N requests, the budget is

$$B(N) = \min(B_{\max}, \max(B_{\min}, \alpha N)), \qquad (1)$$

where α is the budget per request, shared by all of its decode and draft tokens, and $B_{\min}$ and $B_{\max}$ bound the budget

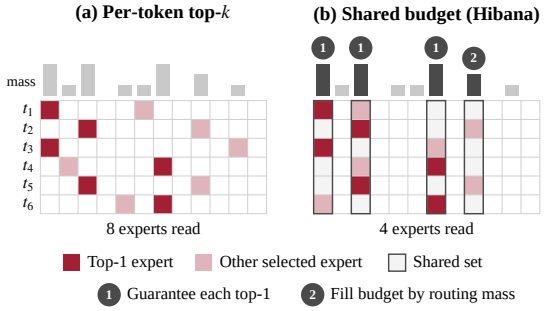

Figure 3. Expert selection in one MoE layer for six tokens with $k = 2$. Bars show each expert's routing mass summed over the tokens that rank it in their top two. (a) Standard routing reads eight expert matrices. (b) Hibana guarantees each token's top-1 expert (step 1), fills the rest of a budget of four with the expert carrying the most routing mass (step 2), and reads four expert matrices.

Table 1. Expert-sharing ablation on Qwen3.8-Flash-Next. g is the number of each token's top experts always kept under a cap of 64 experts per layer. The first row is standard top-10 routing, and the second keeps each token's top five. Accuracy is the paired change on GSM8K, HumanEval, GPQA, and RULER v2 32K relative to the first row. Hibana uses $g = 2$ at these batch sizes.

		Tokens/s		Accuracy
Selection	g	16 req.	64 req.	(pp)
Standard	10	330	702	–
No cap	5	412	765	−0.3
Cap 64	3	488	818	−0.5
Cap 64	2	550	865	−0.8
Cap 64	1	597	984	−1.8

($\alpha = 12$, $B_{\min} = 16$, and $B_{\max} = 64$ on Flash-Next, where each token routes to $k = 10$ of $E = 512$ experts). The budget grows with the number of requests but not with the verification length, and g shrinks as the batch grows so that the guarantees alone do not fill the budget. If the guaranteed experts alone exceed $B(N)$, the layer keeps them, so the budget is a target rather than a hard cap. MoE-Spec (McDanel et al., 2026) caps the experts each layer loads during speculative verification, and Lynx (Gupta et al., 2024) remaps low-affinity token-to-expert assignments within a batch. Unlike both, expert sharing applies one budget to both decode and verification, scales it with the number of requests, and always keeps each token's top experts. With $g = 2$, this costs 0.8 points of accuracy on average (Table 1).

Table 1 shows how we chose these settings for Flash-Next. Without a cap, the union of each token's top five experts reaches 250–460 of the 512 experts per layer at 16–64 requests, so large batches save little. Under a cap of 64, guaranteeing each token its top three experts already exceeds the cap, so the cap rarely applies. A top-two guarantee raises throughput by 23–67% over standard routing, and a top-one guarantee is faster still but loses 1.8 points on average. The minimum budget $B_{\min} = 16$ comes from a single request. With 12 experts, GSM8K fell by 4.7 points, and with 16 by 0.3 points, within noise. Appendix A gives model-specific details.

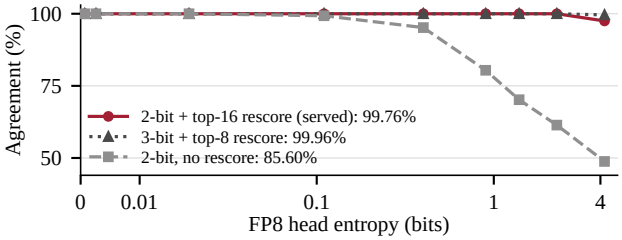

Figure 4. Agreement with the FP8 head's token by the FP8 head's entropy, over 24,460 draft steps of Qwen3.8-Flash-Next. Legend values are overall agreement.

Expert sharing also lowers the marginal cost of verifying more draft tokens, because they can reuse experts already read for the batch. This is codesigned with the adaptive speculation policies in §3.3.1.

3.1.2 *Two-Bit Draft Head*

Hibana drafts with the model's own multi-token prediction (MTP) head (Gloeckle et al., 2024), a small module trained with the target that predicts the next tokens autoregressively from the target's hidden state, or, for Qwen3.8-27B, with DFlash2 (Inco AI, 2026), a diffusion draft model that proposes a block of tokens in one parallel step. Each MTP draft step projects a hidden state onto a vocabulary of about 249k tokens, and reading the FP8 output head for this projection costs about 640 MB per step in terms of memory footprint. Hibana instead ranks every token with a two-bit copy of the head (171 MiB), keeps the 16 highest-scoring candidates, and rescores only those with the matching rows of the target's FP8 head, about 40 KB. The draft emits the candidate with the best FP8 score. The two-bit copy uses the codebook format and kernels that Hibana uses for its three-bit weights (§3.1.3) with four codebook entries instead of eight, the FP8 Lloyd-Max levels of a Gaussian, so each weight costs 2.25 bits including its group scale. Because four levels leave little range, each group's scale is chosen from a few fractions of the group's maximum by least squared error rather than set to the maximum itself.

The majority of the computation runs in two bits, while the final choice uses FP8 scores. Two-bit quantization perturbs each logit slightly, which changes the winner only when the top tokens' scores are very close. These steps benefit from the FP8 head, in which the top token almost always remains among the 16 candidates. Figure 4 confirms this.

Without rescoring, agreement with the FP8 head falls as

the FP8 head's entropy rises, to below 50% on the least certain steps and 85.6% overall. With rescoring, it stays near 100% at every entropy level and reaches 99.8% overall. The target still verifies with its own head, so this approximation changes only which tokens are proposed, not which are accepted. Replaying the recorded rounds yields an unchanged 3.532 accepted tokens per round with the two-bit head and the full draft step of Qwen3.8-Flash-Next at batch one on one DGX Spark decreases from 1.87 ms to 1.55 ms (Appendix B).

3.1.3 *Mixed-Precision Weights*

Hibana stores routed experts, and a decode copy of each dense projection, as three-bit indices into an eight-entry FP8 (E4M3) codebook. Each group of 32 weights has an E4M3 scale and each output row an FP32 scale, so the indices and group scales take 3.25 bits per weight, 72% of NVFP4's 4.5. Hibana fits the indices and scales to calibration activations with GPTQ-style updates (Frantar et al., 2023), then recalibrates the dense decode copies against the original model to preserve quality. Codebook formats for LLM weights have been studied before (Park et al., 2022; Tseng et al., 2024; Guo et al., 2024), and Hibana executes this format on FP8 tensor cores. Its kernels store the packed bits in the order tensor-core instructions consume them, expand them in registers into FP8 operands without a separate dequantization pass, and multiply them by FP8 activations with FP32 accumulation.

The precision depends on the layer and the batch. Three-bit experts replace the NVFP4 copies at every batch size, prefill included, which reduces both memory and weight traffic. Dense projections keep their FP8 or NVFP4 matrices for prefill and add a three-bit copy, at some memory cost, for calls of at most 128 tokens such as decode and verification. Table 2 compares our three-bit MoE kernel with FlashInfer's CUTLASS NVFP4 fused-MoE kernel. It is 1.20–1.36× faster from 512 to 2k tokens, close to the 1.38× ratio of their bits per weight, and 1.06× faster at our 4k-token prefill chunk.

Table 2. Latency of one Flash-Next MoE layer, Hibana's three-bit kernel vs. FlashInfer's CUTLASS NVFP4 fused-MoE kernel.

Tokens	Three-bit (ms)	CUTLASS NVFP4 (ms)	Speedup
512	4.90	6.68	1.36×
1k	5.53	7.22	1.30×
2k	6.84	8.24	1.20×
4k	9.90	10.53	1.06×

3.2 Unified Memory

On a unified-memory device, weights, active requests, saved sessions, and I/O buffers draw on the same memory, and the SSD is the only separate capacity tier. Hibana keeps inactive

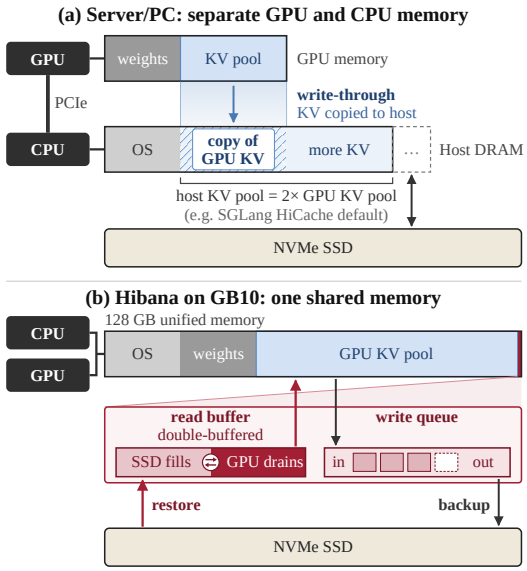

Figure 5. KV-cache tiers on separate versus unified memory. (a) On a server or PC, host DRAM is separate memory, so a host tier that keeps a copy of the GPU's KV adds capacity. (b) On GB10 the CPU and GPU share one memory. Hibana keeps no host copy and moves KV between the GPU's KV pool and the SSD through about 1 GiB of staging, the same at any context length: restores stream through a double-buffered read buffer, and backups wait in a write queue. Only the staging slice is drawn to scale.

sessions on the SSD through a small fixed staging buffer, stores recurrent state in INT8 so that every session takes less memory, and serves large Engram embedding tables (Cheng et al., 2026) from the SSD.

3.2.1 *SSD Session Cache*

To resume an inactive agent, the inference engine needs its KV pages, its recurrent state, and the draft model's state; we call this bundle a *session*. Inactive agent states occupy capacity that active requests need, yet evicting them forces repeated prefill, which is costly especially on local devices. Hibana stores its prefix cache in unified memory (L1) and uses the SSD as an L2 cache. We call the cache that keeps sessions on the SSD the *SSD session cache*.

On a server or PC, the GPU has its own memory, and host DRAM is a separate, usually larger pool. Tiered KV caches therefore use host DRAM as a second tier that holds a copy of the GPU's KV cache; SGLang's HiCache, for example, sizes it at twice the GPU pool by default (SGLang Contributors, 2026) (Figure 5a). On GB10, the CPU and GPU share one 128 GB memory, so Hibana uses the SSD directly as the second tier session cache.

Hibana cannot read the SSD straight into the KV pool. DGX Spark lacks GPUDirect Storage, so the drive transfers data

only to and from host memory, and a stored block is one contiguous run of bytes while its KV is spread over many per-layer pages. Every transfer therefore goes through a staging buffer in host memory, and a GPU kernel moves the data between that buffer and the KV pages.

The staging buffer comes out of the same memory as the KV cache, so Hibana keeps it small and fixed: about 1 GiB, the same at any context length (Figure 5b). The buffer is pinned, so its pages stay in place while the drive and the GPU use it, and all transfers use direct I/O, which bypasses the operating system's file cache instead of keeping a second copy of the data in memory. The buffer is much smaller than a long prefix, so a restore reads the prefix in chunks. With one buffer, the SSD and the GPU would take turns: the drive could not load the next chunk until the GPU had copied the current one into KV pages. Hibana splits the read buffer into two halves (Figure 5b): while the SSD fills one half, the GPU drains the other, and then they swap. The GPU drains a half much faster than the SSD fills one, so the drive rarely waits and a restore runs at close to the drive's read bandwidth. A longer prefix takes more rounds through the same buffer, not a larger buffer. On GB10 the GPU reads the buffer in place, so draining it needs no extra copy into GPU memory. SSD writes are slow, so a GPU kernel first copies KV pages into a write queue, allowing their GPU memory to be reused immediately. Writer threads then save the copies to SSD in the background. The queue has a fixed size to limit memory use. If it is full, evicted pages are discarded without a backup, so requests never wait for SSD writes.

3.2.2 Recurrent-State Quantization

Each recurrent layer of a hybrid model summarizes all earlier tokens in a fixed-size state matrix, which every decode step reads and updates. Hibana keeps a snapshot of this state for every saved session, for every subagent that branches from a shared prefix, and for every speculative candidate under verification, so at full precision these snapshots grow with the number of agents. Hibana therefore stores recurrent state in INT8 using LeapQuant (Pan et al., 2026b), which builds on earlier quantization of state-space models (Chiang et al., 2024; 2025); Nemotron 3 Ultra likewise caches its recurrent state in INT8 (NVIDIA, 2026d). Quantizing the state after every token would compound rounding error, so LeapQuant quantizes it only once per window of tokens. At the start of a window, the state is stored as an INT8 matrix with its scales, and its largest outlier entries are kept separately at high precision as a few compensator tokens that pass through the same update path as real tokens. Within the window, each new token's update is appended to a high-precision buffer instead of being written into the INT8 matrix, and outputs are computed from the INT8 matrix together with the buffered updates. At the end of the window, the buffered updates are folded into the state, which is quantized again, so rounding error enters once per window rather than once per token.

The same buffer makes verification and session saves inexpensive. During verification, the updates of candidate tokens are held in a temporary buffer, and committing a round appends only the updates of the accepted tokens, so Hibana never copies a full recurrent snapshot per candidate and rejected tokens never reach the committed state. A saved session stores the INT8 matrix, its compensator tokens, and the buffered updates together, which also reduces the bytes the SSD session cache writes and restores.

3.2.3 Engram Offloading

Engram tables use the SSD for sparse model-data reads during execution. Hibana keeps the table on the SSD and memory-maps it, so the operating system's page cache holds recently used rows. Each lookup first copies the rows already in the page cache without blocking, then reads the missing rows in parallel from a thread pool, so their SSD latencies overlap instead of adding up. Prefetch overlaps future prompt-chunk or already-proposed-token lookups with available GPU work. Cached rows still consume unified memory, but the whole table need not remain resident; configurations with sufficient capacity can retain their partition in memory. This lookup path is separate from restoring a session and is governed by row locality and the compute available to overlap I/O.

3.3 Adaptive Speculation

Hibana uses one execution lifecycle for multi-token prediction (MTP) heads, autoregressive draft models, and diffusion draft models such as DFlash2 (Inco AI, 2026), which reserves candidate state, drafts, verifies, accepts, commits, and feeds the accepted result back to the draft model. Around this lifecycle, Hibana adapts how much to draft and verify to the expected acceptance (§3.3.1), accounting for how expert sharing (§3.1.1) lowers the cost of verification on MoE targets, and optionally relaxes the acceptance rule (§3.3.2).

3.3.1 Adaptive Drafting and Verification

The objective is to maximize the expected number of emitted tokens per unit of time, including drafting, verification, and commit overhead. An additional draft token is useful only when its chance of being accepted repays the time it takes to draft and verify. Hibana controls this at two points in a round: *adaptive drafting* chooses how many tokens an autoregressive (AR) draft model such as MTP drafts, and *adaptive verification* chooses how many tokens of an already-drafted diffusion block the target verifies.

Adaptive drafting for AR draft models. Prior work adapts the draft length with a learned stopping predictor (Huang et al., 2024) or with dynamic draft trees (Li et al., 2024). Choosing the draft length before drafting avoids both the draft steps and their verification when acceptance is unlikely. For MTP, Hibana maintains a per-request acceptance rate q, an exponentially smoothed ratio of accepted draft tokens to tested positions, where a rejected round counts the accepted tokens and the first rejected position as tested. The desired draft length is

$$d^\star = \arg \max_{1 \le d \le d_{\max}} \frac{\sum_{j=0}^{d} q^j}{1 + c(d-1)}, \tag{2}$$

where $d_{\max}$ is a cap that depends on the batch size. The numerator is the expected number of emitted tokens, including the target's additional token, assuming each position is accepted independently with probability q. The denominator is the round time relative to a round with one draft token, where c is the time that one more draft step and its verification add, as a fraction of that round's time. c is a fixed value configured per model, not measured online.

Hibana computes each request's desired draft length before the round from its completed rounds. Each round runs as a captured CUDA graph, one per combination of batch size and draft length. Small batches use the longest desired draft length among their requests, up to the cap. Larger batches use a fixed draft length per range of batch sizes, which avoids capturing a graph for every draft length at every batch size. All requests in a batch therefore draft the same number of tokens. The caps also bound temporary state and account for weight-dispatch and expert-reuse effects that the simple cost model omits.

Adaptive verification for diffusion drafters. For a diffusion drafter, Hibana adapts how many drafted tokens the target verifies rather than how many it drafts. DFlash2 drafts a whole block in one parallel step, so drafting fewer tokens saves little, while verifying fewer saves target work. EVICT (Pan et al., 2026a) also truncates drafts before verification, aimed at MoE targets where each additional verified token can activate more experts. Hibana instead truncates only on the dense Qwen3.8-27B target, because on MoE targets expert sharing already bounds the experts that verification reads (§3.1.1). For Qwen3.8-27B, the policy predicts how likely each draft token is to be accepted from the drafter's confidence scores and adjusts these predictions with the acceptance observed at each position. A later token counts only if every token before it is accepted, so its expected contribution includes that probability. Using measured latencies of the CUDA graphs that verify each prefix length and an estimate of the remaining round overhead, the policy selects the prefix length for each request that maximizes expected emitted tokens per unit of time. Requests with different prefix lengths are packed into one verification pass, so a confident request verifies more tokens than an uncertain one. Periodic rounds that verify the full block keep the estimates for later positions fresh.

Table 3. Decode speedup of adaptive drafting over a fixed draft length (MTP on Qwen3.8-Flash-Next) and of adaptive verification over verifying the full draft (DFlash2 on Qwen3.8-27B).

Policy	Requests	Speedup
Adaptive drafting	1	1.16×
	4	1.05×
Adaptive verification	8	1.13×
	16	1.28×

This policy applies to greedy batches without grammar constraints. Sampled batches choose a shared verification length before drafting, so the choice does not depend on the sampled draft tokens, and greedy batches containing a grammar-constrained request verify the configured maximum length. Choosing how much to verify is separate from deciding which draft tokens to accept (§3.3.2).

Table 3 shows that adaptive drafting helps most at one request, where the request sets its own draft length, while adaptive verification helps most in larger batches, where verifying every request's full draft is costly.

Adaptive speculation under expert sharing. On MoE targets, expert sharing (§3.1.1) bounds the number of experts a verification pass reads, so verifying more draft tokens adds little cost, and truncating a draft saves little time while discarding tokens that might have been accepted. Hibana therefore verifies Flash-Next's diffusion drafts at a tuned length without truncation, and for Nemotron's MTP head tunes the expert budget together with the draft length, because verifying too many tokens under a fixed budget weakens each token's expert coverage.

3.3.2 Relaxed Verification

Exact speculative sampling can reject a draft token even when the target considers it plausible, reducing the number of tokens produced per round. Medusa's typical acceptance (Cai et al., 2024), Judge Decoding (Bachmann et al., 2025), and loosely speculative decoding (Li et al., 2025) accept such drafts beyond an exact match. We introduce a relaxation parameter $s \ge 0$ to control this tradeoff. Let p and q denote the target and draft sampling distributions at the current history. For an eligible draft token $y \sim q$, the acceptance probability is $a_s(y) = \min(1, e^s p(y)/q(y))$. Setting $s = 0$ recovers exact speculative sampling. Increasing s accepts more proposals, allowing more accepted tokens per round while changing the output distribution. Relaxation is restricted to tokens within the target's top-k predictions

and a configured log-probability margin of its most likely token; other proposals retain the exact acceptance rule. On rejection, we retain the standard residual sampling procedure. A finite s keeps a probability-based check against the target: passing the rank and margin thresholds alone does not guarantee acceptance. As $s \to \infty$, eligible proposals with positive target probability are always accepted. At a fixed history, the resulting next-token distribution P_s satisfies $P_s(x) \leq e^s p(x)$ (Appendix C.1); Appendix C.2 sweeps s. The fast profile uses $s = 0.5$ and treats a draft as eligible when it is among the target's top two tokens and within 1.0 logit of its top token. Table 4 reports the accuracy impact of relaxed verification in the evaluated fast profile, and Figure 8 shows its incremental contribution to decoding speedup. On Qwen3.8-Flash-Next with its MTP drafter, relaxed verification raises the accepted tokens per verify step from 2.15–2.34 under exact verification to 2.76–3.35 at $s = 0.5$, 26–43% more, with no clear loss in accuracy (Appendix C.2).

4 Evaluation

4.1 Experimental Setup

Hardware and targets. All experiments run on one DGX Spark, whose GB10 CPU and GPU share 128 GB of memory (NVIDIA, 2026e). Appendix E reports Hibana on two DGX Sparks. We evaluate four targets, the dense Qwen3.8-27B (Qwen Team, 2026a) and three MoE models, Qwen3.8-Flash-Next (Qwen Team, 2026b), Nemotron-3.5-Bolt (NVIDIA, 2026b), and Nemotron-3.5-Super (NVIDIA, 2026c). Hibana runs its fast profile, which enables all of its optimizations, unless stated otherwise, and its exact profile serves the original model with exact verification.

Baselines. We compare against vLLM (Kwon et al., 2023), llama.cpp (llama.cpp Contributors, 2026a), Flare (NVIDIA, 2026a), and TensorFold (TensorFold Contributors, 2026). Where supported, each baseline runs NVFP4 weights with the fastest drafter we could run for the target, DFlash2 (Inco AI, 2026) for Qwen3.8-27B and the model's MTP head for the others, unless stated otherwise. llama.cpp runs its own quantized weights.

Benchmarks. We measure decoding throughput on SPEED-Bench (Abramovich et al., 2026) with reasoning enabled. We measure end-to-end agentic performance by replaying recorded agent sessions from AgentPerf-Local (Artificial Analysis, 2026a), and we open more agent sessions than GPU memory can hold to stress the cache (§4.3). We measure task accuracy on GPQA (Rein et al., 2023), LiveCodeBench (LCB) (Jain et al., 2024), IFBench (Pyatkin et al., 2025), and AIME 2025 and 2026 (Zhang & Math-AI Team, 2026).

4.2 Decoding Throughput

Figure 6 sweeps concurrency on SPEED-Bench. On all four targets, Hibana reaches higher throughput than vLLM, Flare, TensorFold, and llama.cpp at every concurrency level. Hibana's three-bit weights and compact KV cache leave memory for up to 64 concurrent requests; we run each baseline up to the most it can serve, at most 32. The gain is largest on Qwen3.8-Flash-Next, where Hibana's peak throughput (at 64 requests) is 3.9× that of the strongest baseline's peak (Flare at 32 requests). At the same 32 requests the gap is 3.1×.

4.3 Agentic Tasks

Task throughput. Figure 6b reports end-to-end agentic performance. At each system's best concurrency, Hibana completes 1.5–3.4× more tasks per hour than the fastest baseline. Qwen3.8-Flash-Next gains the most: Hibana keeps rising to 168 tasks per hour at 32 agents, while every baseline stalls or slows past 16.

Task throughput needs both decoding speed and cache reuse. TensorFold decodes faster than vLLM at low concurrency on most targets (Figure 6a), yet from 16 agents on, vLLM completes more tasks per hour on every target (Figure 6b). Some AgentPerf-Local traces rewrite the end of the previous prompt every turn. TensorFold reuses a cached prompt only when the new one extends it exactly, so on Qwen3.8-Flash-Next it serves 0% of these prompts from cache, against 44% for vLLM, which resumes from its last saved point; TensorFold also serves at most 16 streams. Flare would resume from the right point, but its checkpoint pool is fixed and shared. Each request keeps only five checkpoints on Qwen3.8-Flash-Next, so the one before the rewritten end is usually missing, and more agents evict each other's. Its hit rate is 26–56% at one agent and 19% at 32 (Appendix F). Hibana leads on both factors: it decodes fastest at every concurrency (Figure 6a) and keeps 93–96% of these prompts in cache. With the rewrite removed, so all engines reuse context equally well, Hibana still completes about 2× as many tasks per hour as vLLM, which isolates its decoding advantage (Appendix F). On Qwen3.8-Flash-Next, Hibana's lead at 16 agents falls from 3.1× to 1.9× without the turn note.

When sessions outgrow GPU memory. Figure 7 shows that when agent sessions hold more context than GPU memory, only Hibana's SSD tier keeps them cached. We replay 192 AgentPerf-Local conversations at once, so their combined context grows to about twice Hibana's GPU KV capacity. Caches fail here because the conversations take turns: about 130 turns pass before the same conversation returns. If GPU memory cannot hold them all, each one is evicted before it returns, and its next turn recomputes

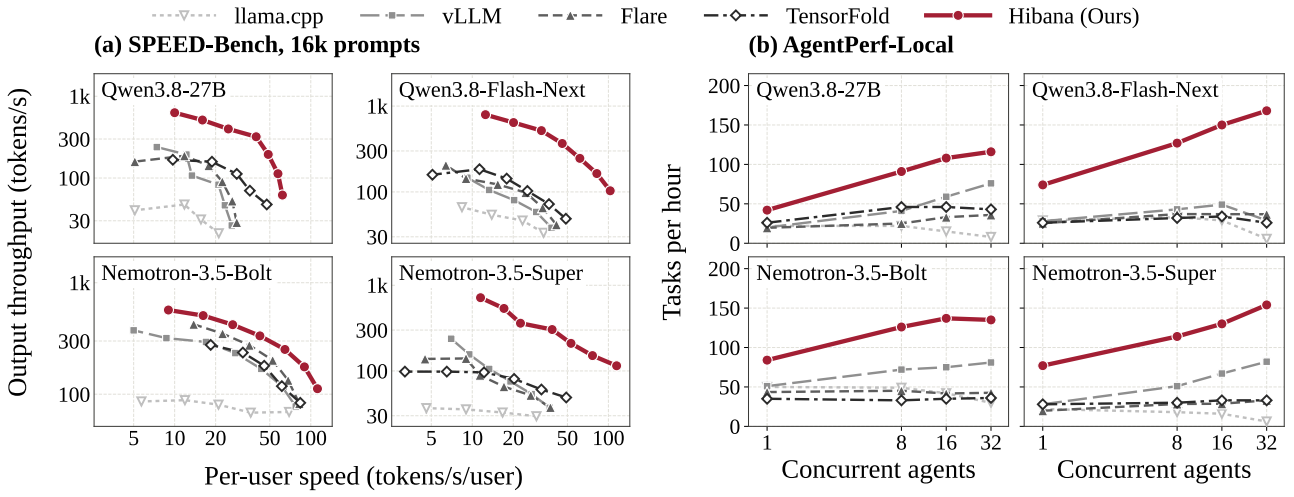

Figure 6. (a) SPEED-Bench throughput on one DGX Spark as concurrency grows. Each point is one concurrency level, labeled with its number of requests. Up and to the right is better. (b) Agentic task throughput on AgentPerf-Local replay as the number of concurrent agents grows. Missing points mark configurations the baseline could not run.

most of its context. On these hybrid models they can keep the saved state of only 16 to 32 conversations, so their hit rates stay below 20%. For vLLM, the limit comes from its page layout: each saved recurrent state takes a KV-cache page enlarged to hold about 3,000 tokens, so few states fit, and a cached prefix is reused only in whole blocks of that size. Hibana without the SSD tier holds more, but its hit rate stays below about 35%, and once live session context passes about 2 M tokens it falls below 10% and each turn recomputes about 11,000 tokens.

The SSD tier removes this limit. Hibana writes each evicted conversation to the SSD and reads it back on the conversation's next turn (§3.2.1). Its hit rate climbs to 80–98%, and each turn recomputes about 600 tokens instead of 11,000. As a result, its median TTFT stays at a few seconds, while it rises to 15–35 s without SSD offloading. The baselines' TTFT exceeds 100 s, up to 42× Hibana's. This larger gap also comes from Hibana's higher throughput, so each turn waits less behind other agents' prefill and decoding. Even when every engine misses the cache, Hibana without the SSD tier has a median TTFT of 25 s, against 210–245 s for the baselines.

4.4 Where the Speedup Comes From

Figure 8 adds Hibana's optimizations one at a time on Qwen3.8-Flash-Next, from a baseline without speculation to the fast profile. Three steps account for most of the gain. MTP drafting with a draft length of 4 raises throughput by 97%, three-bit weights add 43%, and expert sharing adds a further 32%. Adaptive drafting and the final step add 5% each, and the two-bit draft head and relaxed verification add 3% and 1%. In total, throughput rises from 27.9 to 119.6 tokens/s, a 4.3× speedup.

4.5 Accuracy of the Lossy Optimizations

Table 4. Task accuracy (%) on Qwen3.8-Flash-Next as each lossy optimization is added. The last row is the full fast recipe. Parentheses give the change from the exact baseline.

	GPQA	LCB	IFBench	AIME25	AIME26
Exact	90.7	87.7	85.3	100.0	100.0
+ 3-bit weights	90.4 (−0.3)	85.4 (−2.3)	83.3 (−2.0)	99.2 (−0.8)	99.2 (−0.8)
+ Expert sharing	90.4 (−0.3)	85.4 (−2.3)	83.3 (−2.0)	99.2 (−0.8)	99.2 (−0.8)
+ Relaxed verif.	89.1 (−1.6)	84.6 (−3.1)	81.7 (−3.6)	99.2 (−0.8)	99.2 (−0.8)

Exact speculative sampling preserves a fixed target's output distribution (Leviathan et al., 2023), but three-bit weights, expert sharing, and relaxed verification can change it. Table 4 adds these optimizations one at a time on Qwen3.8-Flash-Next. Three-bit weights cost 2.3 points on LCB, 2.0 on IFBench, 0.8 on AIME, and 0.3 on GPQA. Expert sharing leaves every score unchanged. Relaxed verification with $s = 0.5$ costs a further 1.3 points on GPQA, 0.8 on LCB, and 1.6 on IFBench, and leaves AIME26 unchanged. The last row is the complete fast recipe, which also enables the NVFP4 KV cache. It stays within 3.6 points of the exact baseline on every benchmark it was run on. Appendix D reports the same comparison on Nemotron-3.5-Super and Qwen3.8-27B.

5 Related Work

Datacenter serving. vLLM manages the KV cache in fixed-size pages so that many requests can be batched with

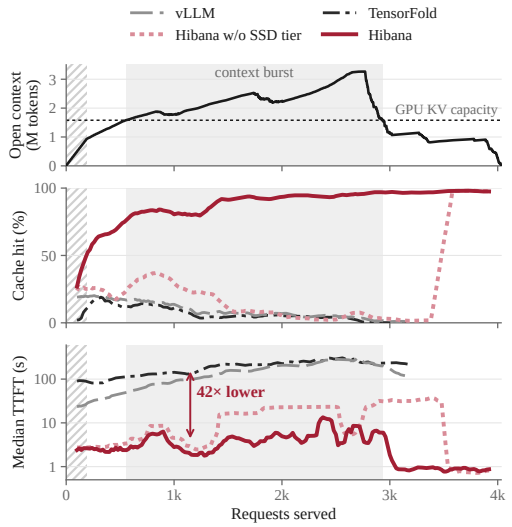

Figure 7. Qwen3.8-Flash-Next serving 192 agent conversations whose combined context reaches twice GPU KV capacity. Top: live session context, the tokens of all unfinished conversations. Middle: share of each prompt served from cache. Bottom: median TTFT, log scale. 32 agents send turns back to back, with thinking off and no tool time. Curves are 200-turn rolling medians; the hatched region is warm-up. Baseline runs were stopped after about 3.3k turns.

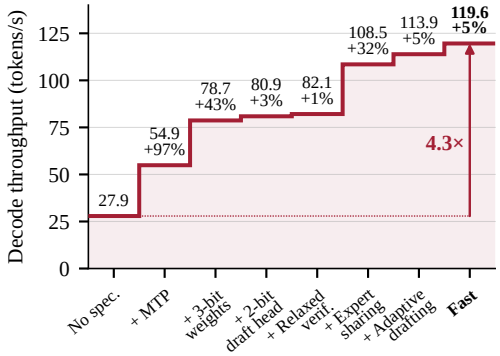

Figure 8. Decode throughput on Qwen3.8-Flash-Next at one request as each optimization is added. Percentages are the change from the previous step. The baseline runs NVFP4 experts without speculation. The last step adds an NVFP4 KV cache.

little wasted memory (Kwon et al., 2023), and SGLang reuses shared prefixes across requests and, with HiCache, extends this reuse to host memory and storage (Zheng et al., 2024; SGLang Contributors, 2026). Both target discrete datacenter GPUs, whereas Hibana applies batching and prefix reuse within the shared memory and limited bandwidth of one unified-memory device.

Local serving. llama.cpp and MLX LM run quantized models on consumer hardware (llama.cpp Contributors, 2026a; Hannun et al., 2023; MLX Contributors, 2026), and llama.cpp's server adds continuous batching, tool calling, and speculative decoding (llama.cpp Contributors, 2026b). NVIDIA Flare targets long conversations and branching agents on DGX Spark and checkpoints attention, recurrent, and speculative state (NVIDIA, 2026a), and TensorFold serves concurrent requests with disk-backed conversation and prefix reuse on Macs (TensorFold Contributors, 2026). Hibana shares their goal of local agent serving but also changes the model and the decoding procedure, so that expert sharing, compressed state, and adaptive speculation reduce the memory traffic behind each accepted token, which limits the benefit of speculative decoding on these devices (Liu et al., 2026).

Memory-constrained execution. FlexGen places and compresses weights across GPU memory, host memory, and disk for batched inference (Sheng et al., 2023), and LLM in a flash loads parameters from flash storage on demand (Alizadeh et al., 2024). FreeToken and Strata bring expert placement, agent-state reuse, and session save and restore to edge MoE serving (Yang et al., 2026; FreeToken Contributors, 2026; Strata Contributors, 2026). Hibana instead keeps execution in unified memory, compresses weights and state to leave room for active sessions, and places only inactive sessions and Engram tables on SSD. Tutti restores SSD-backed KV through GPU-initiated I/O (Qiu et al., 2026). Like Flare and Strata, Hibana saves complete sessions, including attention, recurrent, and draft state, but stores recurrent state in INT8 and restores through a small fixed staging buffer, so persistence takes little memory from active requests.

6 Conclusion

We presented Hibana, an inference engine for agentic workloads on unified-memory devices. Hibana combines model surgery, unified-memory management, and adaptive speculation, and designs them together around the shared memory capacity and limited bandwidth of a unified-memory system-on-chip. On a single DGX Spark serving Qwen and Nemotron models, Hibana completes up to $3.4\times$ more agentic tasks per hour and delivers up to $3.9\times$ higher decoding throughput than vLLM, Flare, llama.cpp, and TensorFold.

References

Abramovich, T., Ashkenazi, M., Putterman, I., Chislett, B., Mitra, T., Rouhani, B. D., Zilberstein, R., and Geifman, Y. SPEED-Bench: A unified and diverse benchmark for speculative decoding. arXiv:2604.09557, 2026. URL https://arxiv.org/abs/2604.09557.

Alizadeh, K., Mirzadeh, I., Belenko, D., Khatamifard, K., Cho, M., Del Mundo, C. C., Rastegari, M., and Farajtabar,

M. LLM in a flash: Efficient large language model inference with limited memory. arXiv:2312.11514, 2024. URL https://arxiv.org/abs/2312.11514.

Artificial Analysis. AA-AgentPerf-Local. GitHub repository, 2026a. URL https://github.com/ArtificialAnalysis/aa-agentperf-local. Accessed October 29, 2026.

Artificial Analysis. Artificial analysis intelligence index: Methodology, 2026b. URL https://artificialanalysis.ai/methodology/intelligence-benchmarking. The trends figure uses version 4.3.2. Accessed October 30, 2026.

Bachmann, G., Anagnostidis, S., Pumarola, A., Georgopoulos, M., Sanakoyeu, A., Du, Y., Schönfeld, E., Thabet, A., and Kohler, J. Judge Decoding: Faster speculative sampling requires going beyond model alignment. arXiv:2501.19309, 2025. URL https://arxiv.org/abs/2501.19309.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. arXiv:2401.10774, 2024. URL https://arxiv.org/abs/2401.10774.

Cheng, X., Tian, R., Zeng, W., Dai, D., Chen, Q., Wang, B., Xie, Z., Huang, K., Yu, X., Deng, C., Zhou, S., Zhao, C., Hao, Z., Li, Y., Zhang, H., Zhang, Z., Wei, Y., Xu, M. Y., Zhang, H., Zhao, D., and Liang, W. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL https://arxiv.org/abs/2601.07372.

Chiang, H.-Y., Chang, C.-C., Frumkin, N., Wu, K.-C., and Marculescu, D. Quamba: A post-training quantization recipe for selective state space models. arXiv:2410.13229, 2024. URL https://arxiv.org/abs/2410.13229.

Chiang, H.-Y., Chang, C.-C., Frumkin, N., Wu, K.-C., Abdelfattah, M. S., and Marculescu, D. Quamba2: A robust and scalable post-training quantization framework for selective state space models. arXiv:2503.22879, 2025. URL https://arxiv.org/abs/2503.22879.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019. URL https://arxiv.org/abs/1904.10509.

Dao, T. and Gu, A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 10041–10071. PMLR, 2024. URL https://proceedings.mlr.press/v235/dao24a.html.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL https://www.jmlr.org/papers/v23/21-0998.html.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv:2210.17323, 2023. URL https://arxiv.org/abs/2210.17323.

FreeToken Contributors. FreeToken: Runtime and supported models. Project documentation, 2026. URL https://github.com/FlashML-org/FreeToken. Includes docs/models.md; accessed October 26, 2026.

Gloeckle, F., Youbi Idrissi, B., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2404.19737.

Gu, Y., Dong, L., Wei, F., and Huang, M. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2306.08543.

Guo, H., Brandon, W., Cholakov, R., Ragan-Kelley, J., Xing, E. P., and Kim, Y. Fast matrix multiplications for lookup table-quantized llms. arXiv:2407.10960, 2024. URL https://arxiv.org/abs/2407.10960.

Gupta, V., Ju, J. H., Sinha, K., Gavrilovska, A., and Iyer, A. P. Lynx: Enabling efficient moe inference through dynamic batch-aware expert selection. arXiv:2411.08982, 2024. URL https://arxiv.org/abs/2411.08982.

Hannun, A., Digani, J., Katharopoulos, A., and Collobert, R. MLX: Efficient and flexible machine learning on apple silicon. GitHub repository, 2023. URL https://github.com/ml-explore/mlx. Accessed October 21, 2026.

Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. URL https://arxiv.org/abs/1503.02531.

Huang, K., Guo, X., and Wang, M. SpecDec++: Boosting speculative decoding via adaptive candidate lengths. arXiv:2405.19715, 2024. URL https://arxiv.org/abs/2405.19715.

Inco AI. DFlash 2: Keep drafting parallel, 2026. URL https://inco.ai/blog/dflash2/.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. arXiv:2403.07974, 2024. URL https://arxiv.org/abs/2403.07974.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Le Scao, T., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and El Sayed, W. Mixtral of experts. arXiv:2401.04088, 2024. URL https://arxiv.org/abs/2401.04088.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. arXiv:2309.06180, 2023. URL https://arxiv.org/abs/2309.06180.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. arXiv:2211.17192, 2023. URL https://arxiv.org/abs/2211.17192.

Li, J., Xu, Y., Li, G., Yang, S., Xu, J., Yin, X., Li, D., Ngai, E. C. H., and Barsoum, E. Training-Free Loosely Speculative Decoding: Accepting semantically correct drafts beyond exact match. arXiv:2511.22972, 2025. URL https://arxiv.org/abs/2511.22972.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster inference of language models with dynamic draft trees. arXiv:2406.16858, 2024. URL https://arxiv.org/abs/2406.16858.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In Chowdhery, A. and Jia, Z. (eds.), *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741. MLSys, 2026. arXiv:2601.11580.

llama.cpp Contributors. llama.cpp: LLM inference in C/C++. GitHub repository, 2026a. URL https://github.com/ggml-org/llama.cpp. Accessed October 18, 2026.

llama.cpp Contributors. llama.cpp HTTP server. Project documentation, 2026b. URL https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md. Accessed October 26, 2026.

McDanel, B., Li, S., Surineni, S., and Khaitan, H. MoE-Spec: Expert budgeting for efficient speculative decoding. arXiv:2602.16052, 2026. URL https://arxiv.org/abs/2602.16052.

MLX Contributors. MLX LM: Run LLMs with MLX. GitHub repository, 2026. URL https://github.com/ml-explore/mlx-lm. Accessed October 26, 2026.

NVIDIA. Flare: A local inference server for NVIDIA SM120/SM121 GPUs. https://github.com/NVIDIA/spark-local-inference, 2026a. Accessed: 2026-11-03.

NVIDIA. NVIDIA-Nemotron-3.5-Bolt-74B-A9B-24GB model card. Hugging Face, 2026b. URL https://huggingface.co/nvidia/NVIDIA-Nemotron-3.5-Bolt-74B-A9B-24GB. Accessed October 29, 2026.

NVIDIA. NVIDIA-Nemotron-3.5-Super-VL-120B-A12B model card. Hugging Face, 2026c. URL https://huggingface.co/nvidia/NVIDIA-Nemotron-3.5-Super-VL-120B-A12B-BF16. Accessed October 28, 2026.

NVIDIA. Nemotron 3 Ultra: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. arXiv:2606.15007, 2026d. URL https://arxiv.org/abs/2606.15007.

NVIDIA. DGX Spark user guide: Hardware overview, 2026e. URL https://docs.nvidia.com/dgx/dgx-spark/hardware.html. Accessed October 20, 2026.

Pan, L., Tao, Z., Wang, X., Zhao, Z., Wang, M., and Zhang, Y. Making Every Verified Token Count: Adaptive verification for moe speculative decoding. arXiv:2605.00342, 2026a. URL https://arxiv.org/abs/2605.00342.

Pan, Y., Xi, H., Zhu, K., Li, X., Wu, Y., Mishra, M., Zhang, H., Zheng, W. X., Kasikci, B., Han, S., Keutzer, K., Iyer, R., and Stoica, I. LeapQuant: Efficient linear attention with accurate recurrent state quantization. arXiv:2609.38166, 2026b. URL https://arxiv.org/abs/2609.38166.

Park, G., Park, B., Kim, M., Lee, S., Kim, J., Kwon, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. LUT-GEMM:

Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models. arXiv:2206.09557, 2022. URL https://arxiv.org/abs/2206.09557.

Pyatkin, V., Malik, S., Graf, V., Ivison, H., Huang, S., Dasigi, P., Lambert, N., and Hajishirzi, H. Generalizing verifiable instruction following. arXiv:2507.02833, 2025. URL https://arxiv.org/abs/2507.02833.

Qiu, S., Hu, Y., Wang, X., Zhu, W., Yan, J., Chen, H., Xu, K., Chen, K., and Zhang, Y. Tutti: Making SSD-backed KV cache practical for long-context LLM serving. arXiv:2605.03375, 2026. URL https://arxiv.org/abs/2605.03375.

Qwen Team. Qwen3.8-27B model card. Hugging Face, 2026a. URL https://huggingface.co/Qwen/Qwen3.8-27B. Accessed October 28, 2026.

Qwen Team. Qwen3.8-Flash-Next model card. Hugging Face, 2026b. URL https://huggingface.co/Qwen/Qwen3.8-Flash-Next. Accessed October 28, 2026.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level google-proof Q&A benchmark. arXiv:2311.12022, 2023. URL https://arxiv.org/abs/2311.12022.

SGLang Contributors. HiCache system design and optimization. SGLang documentation, 2026. URL https://docs.sglang.io/docs/advanced_features/hicache_design. Accessed October 26, 2026.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Fu, D. Y., Xie, Z., Chen, B., Barrett, C., Gonzalez, J. E., Liang, P., Ré, C., Stoica, I., and Zhang, C. FlexGen: High-throughput generative inference of large language models with a single GPU. arXiv:2303.06865, 2023. URL https://arxiv.org/abs/2303.06865.

Strata Contributors. Strata engine and server reference. Project documentation, 2026. URL https://github.com/Niko1221/Strata/blob/main/docs/DETAILS.md. Accessed October 26, 2026.

TensorFold Contributors. TensorFold: Native language model serving on Metal and CUDA. GitHub repository, 2026. URL https://github.com/ashhart/TensorFold/blob/main/README.md. Accessed October 26, 2026.

Tseng, A., Sun, Q., Hou, D., and De Sa, C. QTIP: Quantization with trellises and incoherence processing. arXiv:2406.11235, 2024. URL https://arxiv.org/abs/2406.11235.

Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2412.06464.

Yang, S., Fan, X., Pan, M., Xi, H., Wang, Z., Sun, S., Keutzer, K., Han, S., Zaharia, M., Xu, C., and Stoica, I. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. arXiv:2608.16157, 2026. URL https://arxiv.org/abs/2608.16157.

Zhang, Y. and Math-AI Team. American invitational mathematics examination (aime) 2026. https://huggingface.co/datasets/math-ai/aime26, 2026.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. arXiv:2312.07104, 2024. URL https://arxiv.org/abs/2312.07104.

AI Usage Disclosure

The authors utilized ChatGPT (OpenAI) and Claude (Anthropic) as supporting tools throughout the research and manuscript preparation process.

During the research phase, these tools facilitated brainstorming, exploration of methodological approaches and mathematical formulations, and identification, synthesis, and comparison of relevant literature. During implementation and evaluation, they provided assistance with code development, refinement, and debugging, experimental design, selection of baselines, and interpretation of experimental findings.

For manuscript preparation, the tools assisted with drafting and editing, improving language clarity and grammatical accuracy, developing and refining figures and tables, and reviewing submission guidelines, formatting, and references.

All research decisions, findings, and submitted content were reviewed and validated by the authors, who assume full responsibility for the accuracy, originality, and integrity of the work.

A Expert-Sharing Implementation Details

The expert sharing procedure in §3.1.1 preserves each model's router ranking and weighting conventions. Flash-Next scores an expert by summing its softmax probability over tokens whose original top-k includes it. Nemotron ranks candidates using sigmoid scores plus its expert-selection bias, but aggregates the unbiased sigmoid weights normalized over each token's original top-k. After constructing the shared set, Flash-Next selects up to k eligible experts from a wider ranked candidate list; Nemotron retains eligible experts from its original top-k.

In the request-scaled configuration, Flash-Next uses a minimum budget of 16 experts; Nemotron has no additional minimum. For example, $\alpha = 12$, $B_{\min} = 16$, and $B_{\max} = 64$ yield budgets of 16, 24, 48, and 64 experts for 1, 2, 4, and 8 real requests. These are configured budgets, not measured expert counts. Flash-Next decreases the per-token guarantee as the execution shape grows. Nemotron applies a smaller fallback guarantee if the initial guaranteed union exceeds the budget. Both preserve the resulting guaranteed set even if it exceeds the cap. Guarantee sizes are derived from the execution shape, including graph-bucket dimensions; the shared-set budget counts only real requests.

B Two-Bit Draft Head Agreement

Figure 4 compares the two-bit draft head with and without FP8 rescoring (§3.1.2) on 24,460 recorded greedy draft steps of Qwen3.8-Flash-Next in the fast profile. Each step is binned by the entropy of the FP8 head's output distribution, and each curve reports how often it selects the same token as the full FP8 head.

On confident steps, below about 0.1 bits of entropy, both variants agree with the FP8 head, because the top token leads by a margin larger than the two-bit quantization error. As entropy rises, the leading tokens score more closely and the two-bit head alone increasingly picks a different winner, falling to about 80% agreement near one bit and below 50% at four bits. Rescoring removes this loss. The two-bit ranking still places the FP8 winner among its 16 highest-scoring candidates, so recomputing those 16 scores with FP8 rows recovers it. The served two-bit head with 16 candidates agrees on 99.76% of steps overall, close to the earlier three-bit head with 8 candidates at 99.96%. Replaying the recorded rounds yields 3.532 accepted tokens per round for both.

The rescoring pass reads only 16 rows of the FP8 head, under 0.01% of the full head. On one DGX Spark, the head projection decreases from 1.11 ms with the three-bit head to 0.78 ms with the two-bit head and rescoring, which accounts for the 1.87 to 1.55 ms reduction in the full draft step reported in §3.1.2.

C Relaxed Verification

Relaxed verification (§3.3.2) accepts an eligible draft token y with probability $\min(1, e^s p(y)/q(y))$, where $s \geq 0$ is the relaxation parameter and $s = 0$ recovers exact speculative sampling. This appendix bounds how much this rule can amplify any token's probability (§C.1) and measures how accuracy and acceptance change with s (§C.2).

C.1 Output Distribution

Fix a decoding history and let p and q be the normalized target and draft distributions used by the sampling procedure, including its temperature and filtering settings. Let S be the eligible token set, the tokens that pass the rank and margin test of §3.3.2; S depends only on p, so each c_x below is fixed at a given history. We define $c_x = e^s$ for $x \in S$ and $c_x = 1$ otherwise, for finite $s \geq 0$. The probability mass emitted by accepting a proposal x is

$$m_s(x) = \min\{q(x), c_x p(x)\}, \qquad \alpha_s = \sum_x m_s(x). \quad (3)$$

After rejection, the sampler uses the unchanged residual

$$r(x) = \frac{[p(x) - q(x)]_+}{Z}, \quad (4)$$

where $Z = \sum_x [p(x) - q(x)]_+$ and $[z]_+ = \max(z, 0)$. Thus, for $Z > 0$, the next-token law is

$$P_s(x) = m_s(x) + (1 - \alpha_s) r(x). \quad (5)$$

If $Z = 0$, then $p = q$, every proposal is accepted, and $P_s = p$ without residual sampling.

Bound on probability amplification. Since $c_x \geq 1$, $\alpha_s \geq \sum_x \min\{p(x), q(x)\} = 1 - Z$. For $q(x) < p(x)$, $m_s(x) = q(x)$ and $1 - \alpha_s \leq Z$, so $P_s(x) \leq q(x) + p(x) - q(x) = p(x)$. For $q(x) \geq p(x)$, the residual is zero and $P_s(x) = m_s(x) \leq c_x p(x)$. Together these give $P_s(x) \leq c_x p(x) \leq e^s p(x)$ for every token. At $s = 0$, Equation 5 reduces to $P_0 = p$. The bound compares distributions at the same history and for the fixed target used in verification; it does not establish sequence-level or task-quality equivalence to the original uncompressed model.

Deterministic draft proposals. For a draft model that deterministically proposes y, $q(y) = 1$, so an eligible proposal is accepted with probability $\min\{1, e^s p(y)\}$. It is accepted unconditionally only when $p(y) \geq e^{-s}$. This case covers the point-mass MTP proposals in the sampled verification path; it is distinct from greedy target decoding.

	Accuracy (%)					Accepted tokens per verify step				
Acceptance	GPQA	LCB v6	IFBench	AIME26	Avg.†	GPQA	LCB v6	IFBench	AIME26	Capped‡
Exact	87.4	83.7	82.0	99.6	84.4	2.25	2.15	2.34	2.30	7
Relaxed, s=0.1	89.7	84.6	85.7	100.0	86.7	2.40	2.32	2.55	2.43	6
Relaxed, s=0.25	88.9	84.3	84.8	99.2	86.0	2.60	2.49	2.78	2.63	7
Relaxed, s=0.5	89.1	84.6	81.7	99.2	85.1	2.86	2.76	3.35	2.89	26
Relaxed, s=1	87.7	85.7	76.3	97.5	83.2	3.25	3.19	3.72	3.21	55

Table 5. Qwen3.8-Flash-Next with its MTP drafter (3 drafted tokens per step), three-bit weights (§3.1.3) and expert sharing, under each acceptance rule. Under relaxed verification, an eligible draft y (in the target's top 2 and within 1.0 logit of its top token) is accepted with probability $\min(1, e^s p(y))$; other drafts get the exact test. 248K-token cap, thinking on, T=1.0, top-p 0.95, top-k 20

C.2 Accuracy and Acceptance Across s

Table 5 sweeps s on Qwen3.8-Flash-Next with its MTP drafter, three-bit weights, and expert sharing. In short, $s \leq 0.5$ yields 6–43% more accepted tokens per verify step with no clear accuracy cost. The fast profile uses $s = 0.5$ (Table 4), the most relaxed of these settings, while $s = 1$ costs accuracy. The bound in §C.1 is a worst case: it caps the factor by which any token's probability can rise at e^s, about 1.11, 1.28, 1.65, and 2.72 for the four settings, but it does not predict task accuracy, which we measure directly.

Acceptance rises steadily with s. Relative to the exact rule, accepted tokens per verify step increase by 6–9% at $s = 0.1$, 14–19% at $s = 0.25$, 26–43% at $s = 0.5$, and 40–59% at $s = 1$, with the largest gain on IFBench at every setting.

Accuracy holds up to $s = 0.5$. At $s = 0.1$, $s = 0.25$, and $s = 0.5$, every benchmark scores above the exact rule or within 0.4 points of it, and the average of GPQA, LiveCodeBench, and IFBench rises from 84.4% to 86.7%, 86.0%, and 85.1%. These are means over two to eight seeds, and the bound permits no systematic gain, so we read them as no clear loss rather than as a gain. Accuracy first degrades at $s = 1$, where IFBench falls 5.7 points and AIME26 2.1 points, and the average drops below the exact rule. More answers also reach the 248K-token cap as s grows: 6–7 of 1,784 up to $s = 0.25$, then 26 at $s = 0.5$ and 55 at $s = 1$.

D Accuracy on Other Targets

Table 6 compares the exact profile, three-bit weights alone, and the full fast profile on Nemotron-3.5-Super, Qwen3.8-Flash-Next, and Qwen3.8-27B.

E Two DGX Sparks

Hibana can also serve one model across two DGX Sparks connected by a RoCE link, using tensor parallelism. Attention, recurrent, and dense projection layers are split by head or by row, and the three-bit routed experts are split by expert, so each Spark holds half of the experts whole. Table 7 reports the resulting decode speedup on Qwen3.8-Flash-Next, which grows from 1.56× at one request to 1.80× at 32 requests.

Table 6. Task accuracy (%) at a 248k-token budget. Fast is the full fast profile with relaxed verification. Avg. is over all five tasks, except for Flash-Next, where it leaves out the saturated AIME sets.

	GPQA	LCB	IFBench	AIME25	AIME26	Avg.
Nemotron-3.5-Super						
Exact	87.2	86.5	75.2	95.8	94.6	87.9
3-bit	86.7	82.6	71.3	94.2	95.8	86.1
Fast	87.7	79.7	70.2	97.1	94.6	85.8
Qwen3.8-Flash-Next						
Exact	90.7	87.7	85.3	100.0	100.0	87.9
3-bit	89.6	85.4	83.3	99.2	99.2	86.1
Fast	87.0	86.0	81.7	97.5	97.5	84.9
Qwen3.8-27B						
Exact	89.2	84.3	84.5	99.6	98.8	91.3
3-bit	86.0	87.1	82.3	89.6	96.2	88.3
Fast	86.5	86.9	82.3	92.5	95.4	88.7

Table 7. Decode throughput of Qwen3.8-Flash-Next on two DGX Sparks relative to one. At 16 and 32 requests, Hibana uses the fast profile's optimizations with a larger batch limit and shorter drafts.

Requests	Two Sparks vs. one
1	1.56×
4	1.59×
16	1.79×
32	1.80×

F Context Reuse per Trace

This appendix explains why the engines reuse different amounts of context in the AgentPerf-Local replay (§4.3). We replay the eight default tasks on Qwen3.8-Flash-Next with one agent, so nothing is evicted and every cache miss comes from where an engine can resume.

Where an engine can resume. Attention layers keep one KV entry per token, so an engine can keep any shared prefix and drop the rest. Flash-Next's recurrent layers instead keep

one running state that every token updates, so an engine can resume only at a token where it saved a copy of that state. Each copy is about as large as 2,992 tokens of KV cache, so engines save only a few. vLLM saves the leading instructions, the edge of a 4,096-token prefill chunk, and the end of each request. TensorFold resumes only from the exact end of the previous request. In Flare's public code (commit `6fa01a49`), the pool of saved states is fixed and shared by all requests. Each request gets an equal share, five on Flash-Next, and when the pool fills, Flare first evicts the saved states closest to their neighbours. Hibana also saves the start of the last message, the point an agent's next turn usually keeps.

The SWE-bench turn note. The three SWE-bench tasks end every request with a "Tool-call turn k of 100" message and delete it on the next turn; the note is part of the published traces. Every end-of-request save point therefore includes a note that the next prompt no longer has. Figure 9 walks through one such turn, and Figure 10 (top) shows the result on every turn. TensorFold serves nothing from cache, vLLM's hit rate decays as the context grows and jumps only when its chunk-edge point moves up by 4,096 tokens, and Hibana stays at 85–98%. With only five checkpoints per request, Flare, like vLLM, often has none near the prompt end, so it falls back thousands of tokens and jumps only when one lands close. With room in its pool it does keep the boundary before the note: a fresh Nemotron-3.5-Bolt server serves 94–99% of django-14500 from cache, and 86% once the eight tasks fill the pool and evict that checkpoint. With 32 agents sharing the pool (the scaling run of Figure 6b, a different run from the one-agent table below), Flare serves 41% of these prompts from cache on Nemotron-3.5-Bolt and 19% on Qwen3.8-Flash-Next. At 32 agents its GPU is fully busy but decodes only 19–29% of the time, and it recomputes 1.5–2.1× vLLM's prompt tokens (Qwen3.8-27B, Nemotron-3.5-Bolt, and Nemotron-3.5-Super). In our runs, llama.cpp stays close to Hibana. On the append-only OpenClaw tasks, the engines reuse similar amounts of context on the three longer tasks. On the two five-turn tasks, TensorFold caches more (92% against 78% on `k8s-debugging`), and on `skill-search` Hibana and TensorFold both do (94% and 96% against 78%) (Table 8).

Removing the per-turn prompt edit To confirm that the note alone causes these misses, we replayed the same traces with the 108 note messages deleted, so every task only appends. All engines then follow the same line (Figure 10, bottom): vLLM caches 93% of each prompt over all eight tasks and TensorFold 86–93%, while Hibana stays at 92%. Flare rises from 26–56% to 93–95% on the SWE-bench tasks. Hibana still completes about 2× as many tasks per hour as vLLM (77 against 37 at one agent, 154 against 82 at 16), and this remaining gap comes from decoding speed: about 113 tokens/s against vLLM's 46 at one agent, from a fit of each turn's time to its output tokens and uncached prompt tokens. The same fit puts prefill at 13% of vLLM's one-agent time and 16% of Hibana's, so decoding explains almost all of the gap. Flare's tasks per hour at 32 agents rise 1.7–2.5× (37 to 80 on Flash-Next). Without the note, vLLM's cache share on Flash-Next rises from 70% to 93% and its tasks per hour at 16 agents from 49 to 82, while the other three targets are unchanged. Setup details. The replay runs AgentPerf-Local's eight recorded tasks (168 turns) in a closed loop. Each of N agents takes one task and sends its turns back to back with no tool time. Every request carries the recorded conversation and tool schemas, with thinking off, and generates the recorded output length, so all engines do the same work on token-identical prompts. We discard each engine's reply, so a task here is a replayed task, not a solved one. Tasks per hour is tasks finished over wall time, and at eight or more agents the slowest task sets the wall time. On Flash-Next, vLLM and Flare serve NVIDIA's FP4-Local checkpoint and Hibana serves RadixArk's NVFP4 build. The replay uses Hibana's throughput profile, which keeps exact verification only on Nemotron-3.5-Bolt. At 32 agents on Flash-Next, vLLM's cache share falls to 29% and its 7.4M uncached prompt tokens take most of a 4,027 s run, against 0.8M for Hibana. The replay differs from Artificial Analysis's own setting, which leaves thinking at the server default. The burst study's TTFT counts only replies streamed in more than three chunks over at least 50 ms, because Hibana returns tool-call replies in one piece.

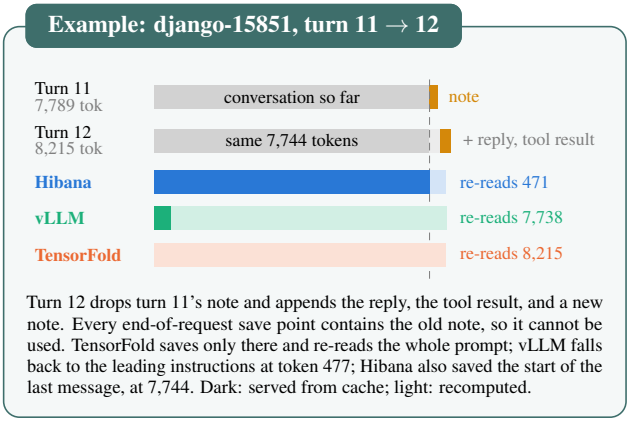

Figure 9. Where each engine resumes on one turn of a SWE-bench trace, drawn to scale.

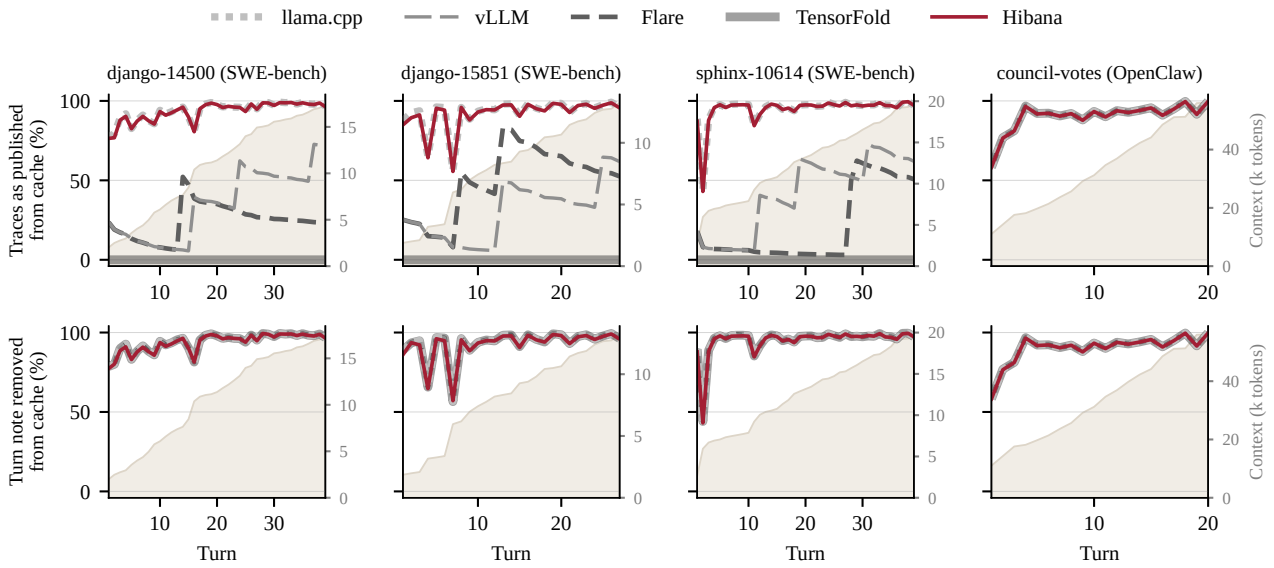

Figure 10. Share of each prompt served from cache on every turn of four AgentPerf-Local tasks, Qwen3.8-Flash-Next, one agent. Gray: the prompt's length. Top: traces as published, where each SWE-bench request ends with a turn note that the next turn deletes. Bottom: the same traces with the note removed. The first turn of each task is omitted. Flare runs its recipe defaults; Table 8 omits llama.cpp.

Table 8. Context reuse per task, traces as published, Qwen3.8-Flash-Next, one agent. Re-read is the prompt tokens an engine recomputes per turn, excluding each task's first turn.

			Served from cache (%)				Re-read per turn (tokens)			
Task	Kind	Turns	vLLM	Hibana	TensorFold	Flare	vLLM	Hibana	TensorFold	Flare
django-14500	SWE-bench	40	44	95	0	26	6,018	473	10,718	7,893
django-15851	SWE-bench	28	36	93	0	56	5,267	499	8,210	3,595
sphinx-10614	SWE-bench	40	49	96	0	26	6,482	507	12,708	9,345
council-votes	OpenClaw	21	93	92	93	93	2,483	2,520	2,490	2,483
council-budget	OpenClaw	20	91	91	91	91	1,991	2,020	1,998	1,991
searchable-index	OpenClaw	9	87	87	87	87	2,784	2,825	2,791	2,784
k8s-debugging	OpenClaw	5	78	78	92	78	410	446	417	410
skill-search	OpenClaw	5	78	94	96	78	281	316	288	281